

Overview:

The total codebase consists of 6 classes, three operate as main methods used for running specific parts of the code and provide some output to the user, and three create objects that store the *SA Place Data*.

PlaceNameEntry:

- Saves data of a single place, id, name, etc.
- Has multiple constructors allowing creation from an array or individual elements, as well as having a copy constructor
- Used by PlaceNameArray and PlaceNameBST as both classes store instances of PlaceNameEntry as their values

PlaceNameArray:

- Array that dynamically stores instances of the PlaceNameEntry class not allowing for duplicates
- Allows the user to continuously load additional PlaceNameEntries and to search for a specific entry using the name of the place
- Is run by the user in the PlaceSearchArray class and is also automatically used by the PlaceExperiment class

PlaceSearchArray:

- Allows the user a set of actions that can be done to an instance of PlaceNameArray. Load, Search, Help(Creative Extension), and Quit. (case insensitive)
-

PlaceNameBST:

- BST that dynamically stores instances of the PlaceNameEntry class not allowing for duplicates
- Allows the user to continuously load additional PlaceNameEntries and to search for a specific entry using the name of the place
- Is run by the user in the PlaceSearchBST class and is also automatically used by the PlaceExperiment class

PlaceSearchBST:

- Allows the user a set of actions that can be done to an instance of PlaceNameBST. Load, Search, Help(Creative Extension), and Quit. (case insensitive)
-

PlaceExperiment:

- The program runs an experiment comparing the number of comparisons made when searching for an entry in an array, bst, bst(sorted) and bst(optimal) with entries from 1000 to 10000 in increments of 1000.

PlaceSearchArray:

```
lobst@1ArsPC1:~/workspace/uct/csc2001f/Assignment1$ java PlaceSearchArray
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
L
Please enter the path to the file: files/SAplaceNames.csv
Please enter how many places to load: 36
36 places loaded.
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
S
Please enter the name of the place to find: Citrusdal
---Searching for: Citrusdal---
Place not found in database
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
S
Please enter the name of the place to find: Citrusdal
---Searching for: Citrusdal---
161007 Citrusdal Cederberg Western Cape 7177
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
H
- PlaceSearchArray:
Program allows the user to dynamically load Place data into an array
and then lets the user to search the array for additional information about a place after providing the name
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
Q
lobst@1ArsPC1:~/workspace/uct/csc2001f/Assignment1$ |
```

PlaceSearchBST

```
lobst@1ArsPC1:~/workspace/uct/csc2001f/Assignment1$ java PlaceSearchBST
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
L
Please enter the path to the file: files/SAplaceNames.csv
Please enter how many places to load: 24
24 places loaded.
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
S
Please enter the name of the place to find: Citrusdal
---Searching for: Citrusdal---
161007 Citrusdal Cederberg Western Cape 7177
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
S
Please enter the name of the place to find: Cape Town
---Searching for: Cape Town---
Place not found in database
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
H
- PlacesSearchBST
Program allows the user to dynamically load Place data into a binary search tree
and then lets the user to search for additional information about a place after providing the name
What would you like to do? (L)oad (S)earch (H)elp (Q)uit
Q
lobst@1ArsPC1:~/workspace/uct/csc2001f/Assignment1$ |
```

Edge Cases:

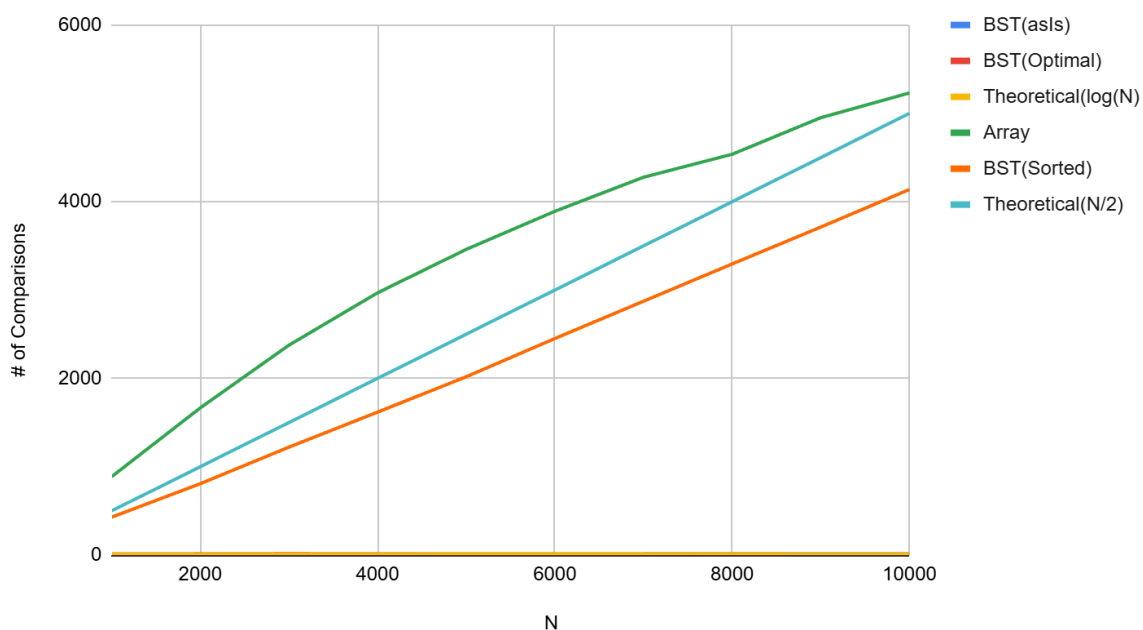
- Searching for last or first element
- Searching for not existent element
- Searching an empty instance
- Searching with very small N
- Searching with very large N

Data Table:

N	Array	BST(asls)	BST(Sorted)	BST(Optimal)
1000	886.42	12.64	428.5	13.6
2000	1665.76	13.74	807.7	14.52
3000	2375.64	14.42	1221.58	15.8
4000	2968.46	14.9	1617.9	14.94
5000	3460.4	15.22	2018.4	14.48
6000	3891.42	15.32	2451.34	14.6
7000	4277	15.6	2873.36	14.74
8000	4537.36	15.8	3295.5	14.68
9000	4951.3	15.9	3713.46	14.3
10000	5231.3	15.96	4138.6	14.18

Complete Graph:

BST(asls), BST(Optimal), Theoretical(log(N), Array, BST(Sorted)...



As can be seen in the graph the both the array and the BST(sorted) had an increase of search cost of $O(N)$ meanwhile both the BST(asls) and the BST(optimal) had an increase of search cost equal to $\log(N)$. The BST(sorted) has a search cost of $O(N)$ as inputting a sorted data set into a non balancing tree causes all the new node to be inserted on the right of the previous node which creates what is essentially a linked list causing searching to be $O(N)$. Meanwhile BST(optimal) slightly outperforms BST(asls) due to the optimal ordering creating a tree that is perfectly balanced and thus producing the smallest average search times as the data grows. These results match the theory as searching an array or linked list is $O(N)$ and searching a mostly balanced tree is $O(\log(N))$.

